code

Definitive Documentation: The Lunar Elemental Mapping Pipeline

Project Objective

To process raw X-ray data from the Chandrayaan-2 CLASS and XSM instruments, perform a full scientific analysis, and produce a high-fidelity map of elemental abundance ratios (Mg/Si, Al/Si) on the lunar surface.

Core Libraries Used

- `astropy` & `pandas`: For reading, writing, and manipulating FITS and CSV data files.
- `numpy` & `scipy`: For numerical operations, including spectral fitting and interpolation.
- `xspec` (`pyxspec`): The professional astrophysics software used for high-level spectral analysis (forward-fitting).
- `matplotlib` & `healpy`: For scientific plotting and creating the final equal-area maps.

Phase 1: Data Preparation & Filtering

This phase transforms thousands of raw instrument files into a single, clean list of high-quality observations that are ready for scientific analysis.

Step 1.1: Extract CLASS Metadata

- **Script:** `perMonthEachDay_extract_class_metadata.py`
- **Objective:** To catalog the time, location, and geometry of every lunar observation made by the CLASS instrument.
- **Inputs:** Raw CLASS `.fits` files for a given month, organized by date.
- **Procedure:** The script efficiently loops through all `.fits` files. It reads only the file headers (not the full data) to extract key metadata like observation times

(`STARTIME`, `ENDTIME`), satellite position (`SAT_LAT`, `SAT_LON`), the instrument's footprint center (`BORE_LAT`, `BORE_LON`), the footprint's four corner vertices (`v0_lat`, `v0_lon`, etc.), and the solar illumination angle (`SOLARANG`).

- **Output:** A series of daily CSV files (e.g., `metadata_CLASS_01.csv`), with each row representing a single CLASS observation.

Step 1.2: Extract XSM Metadata

- **Script:** `new_extract_xsm_metadata.py`
- **Objective:** To catalog the per-second brightness and spectral information from the XSM instrument, which continuously monitors the Sun.
- **Inputs:** Raw daily XSM `.fits` file.
- **Procedure:** The script reads the large binary table within the XSM FITS file. It iterates through each of the ~86,400 rows (one for each second of the day), extracts the `UTCString` timestamp, and calculates summary statistics (e.g., `TotalCounts`) from the `DataArray` column, which contains the full spectrum for that second.
- **Output:** A detailed, per-second CSV file for the day (e.g., `metadata_XSM_ch2_xsm_20250801_v1_level1.csv`).

Step 1.3: Temporally Match CLASS & XSM Data

- **Objective:** To link each brief CLASS lunar observation to the precise solar conditions that occurred during that exact moment.
- **Script:** `match_CLASS_XSM.py`
- **Inputs:** The daily CLASS and XSM metadata CSVs from the previous steps.
- **Procedure:** The script reads both CSV files. For each CLASS observation's time window (typically 4-8 seconds), it finds all the 1-second XSM data points that fall within that window. It then calculates the average solar flux (`xsm_mean_totalcounts`) during that period and merges this information into a new, combined table.
- **Output:** A new, combined CSV file for the day (e.g., `match_CLASS_XSM_20250801.csv`).

Step 1.4: Filter for High-Quality Observations

- **Objective:** To remove scientifically unusable data based on instrument health and observation geometry.
- **Script:** `diagnose_and_filter_match_v1.2.py`
- **Inputs:** The matched CSV file from Step 1.3.
- **Procedure:** This script applies a series of quality-control filters. It keeps only the data rows where key values are within acceptable scientific ranges, such as a solar angle less than 90° (to ensure the surface is sunlit), a specific satellite altitude range (to ensure consistent footprint size), and stable detector temperatures.
- **Output:** A `..._filtered.csv` file containing only scientifically valid observations.

Step 1.5: Create Premium Observation Subset

- **Objective:** To isolate the absolute best-quality data that will produce the highest signal-to-noise results.
- **Script:** `filteredFiles_subset_basedOnSolarAng.py`
- **Inputs:** The `..._filtered.csv` file from Step 1.4.
- **Procedure:** The script creates a final, smaller CSV file containing only observations with the best possible illumination conditions (`solarang_deg <= 60.0`).
- **Output:** The final input for the main analysis pipeline: `match_CLASS_XSM_20250801_prime.csv`.

Phase 2: Automated Scientific Analysis Pipeline

This phase is controlled by a single master script that automates the entire scientific analysis for every observation in the `...prime.csv` file.

- **Objective:** To derive the final elemental abundance ratios (Mg/Si, Al/Si) for every single high-quality observation.
- **Main Script:** `run_analysis_batch.py`

Step 2.1: The Batch Process

- **Procedure:** The `run_analysis_batch.py` script loops through every row of the `...prime.csv` file. For each row, it executes the following four-step scientific pipeline in memory:

Sub-step 2.1.1: Prepare XSM Spectrum

- **Helper Script:** `prepare_xsm_for_xspec.py`
- **Procedure:** This function co-adds the individual 1-second XSM spectra that correspond to the CLASS observation, rebins the resulting 2048-channel spectrum down to 512 channels to match the calibration files, and creates a temporary `.pha` FITS file with a complete, OGIP-compliant header.

Sub-step 2.1.2: Calibrate Solar Spectrum (Forward-Fitting)

- **Method:** The `run_xspec_analysis_with_pyxspec` function.
- **Procedure:** This is a pure **forward-fitting** step using the `pyxspec` library. It loads the `.pha` file and its calibration files (`.rsp`, background). It defines a physical model for the Sun's emission (`apec` model), which `XSPEC` then **forward-folds** through the instrument's response. The function adjusts the model's parameters (like plasma temperature) until the simulated spectrum matches the observed XSM data. The final, "unfolded" solar spectrum (`F_sun(E)`) is extracted directly into NumPy arrays.

Sub-step 2.1.3: Isolate the Lunar Signal (Model-and-Subtract)

- **Method:** The `run_continuum_subtraction` function.
- **Procedure:** This function loads the corresponding CLASS spectrum. It builds a physical model for the scattered solar continuum using the `F_sun(E)` from the previous step. It fits this contamination model to the parts of the CLASS spectrum that are known to be free of elemental lines. Finally, it **subtracts** this best-fit continuum model from the total CLASS signal.

Sub-step 2.1.4: Measure Elemental Fingerprints (Fitting)

- **Helper Script:** `py_fit_xrf_lines.py`
- **Procedure:** This function takes the clean residual spectrum from the previous step. It defines a model of three Gaussian peaks and fits it to the known

energies of the Mg, Al, and Si lines to measure the area (flux) of each peak. The final Mg/Si and Al/Si ratios are calculated.

- **Final Output of Phase 2:** A master CSV file, `final_abundance_ratios.csv`, containing the measured ratios and coordinates for every successfully processed observation.

Phase 3: Final Map Generation

This final phase visualizes the scientific results from the batch analysis.

Step 3.1: Consolidate Data

- **Objective:** To merge the scientific results with the full footprint area data.
- **Script Used:** `merge_results.py`
- **Inputs:** `final_abundance_ratios.csv` and `...prime.csv`.
- **Procedure:** Merges the two CSV files on a common observation ID to create a single master file with both the abundance ratios and the `v0..v3` vertex coordinates.
- **Output:** `final_abundance_ratios_complete.csv`.

Step 3.2: Create the Final Map

- **Objective:** To create a scientifically accurate, area-preserving map of the elemental abundances that correctly handles overlapping data.
- **Script Used:** `plot_healpix_map.py`
- **Procedure:** This script uses the `healpy` library to create an equal-area grid of the lunar sphere (**HEALPix**). For each observation, it bins five points (the four corners and the center of the footprint) into this grid. It then calculates the average abundance ratio for each grid pixel, automatically handling the averaging of all overlapping observations. Finally, it plots this grid using a **Mollweide projection**, which accurately represents the relative areas on a sphere.
- **Output:** The final map images (e.g., `lunar_healpix_map_advanced_mg_si_ratio.png`).